

Dynamic Array Implementation

Subject: Data Structures Topic: Arrays

Overview

Deep dive into dynamic arrays, resizing strategies, and amortized analysis

Representation

- A dynamic array stores a pointer to a fixed-capacity buffer plus its current length and capacity.
- Appending is $O(1)$ while spare capacity remains; a full buffer must allocate a larger block and copy elements.
- A common growth policy doubles capacity, balancing occasional copies against unused reserved space.

Amortized analysis

- With doubling, the copies across n appends are bounded by a geometric sum: $1 + 2 + 4 + \dots < 2n$.
- Therefore a sequence of n appends costs $O(n)$ total and $O(1)$ amortized per append, although an individual resize costs $O(n)$.
- Amortized cost is a sequence-wide guarantee; it is not the same as average-case probability.

Trade-offs and operations

- Appending is amortized $O(1)$, indexed access is $O(1)$, and inserting or deleting near the front remains $O(n)$.
- Growing by a factor near one causes frequent copies; very large factors waste more memory.
- Shrinking should use a lower threshold than the growth threshold to prevent repeated grow-shrink oscillation.

Implementation checks

- Before allocation, guard against capacity overflow and allocation failure.
- Copy exactly the live elements, update the pointer and capacity only after successful allocation, then release the old buffer.
- Test empty, full, resize, and repeated append cases.